

TEMA 1

INTRODUCCIÓN A LAS TECNOLOGÍAS WEB: EL PROTOCOLO HTTP

APLICACIONES WEB

Objetivo del tema: Comprender cómo funciona HTTP, el protocolo que conecta a navegadores y servidores en la Web.



Esta obra está bajo una
Licencia CC BY-NC-SA 4.0 Internacional.

Jesús Cáceres Tello
Dpto de Sistemas Informáticos y Computación
Facultad de Informática
Universidad Complutense de Madrid

CONTENIDO

- REPASO DEL MODELO TCP/IP
- MODELO CLIENTE/SERVIDOR
- SOCKETS Y PROTOCOLOS
- PROTOCOLO HTTP
- REFERENCIAS



EL MODELO OSI: Cómo viajan los datos

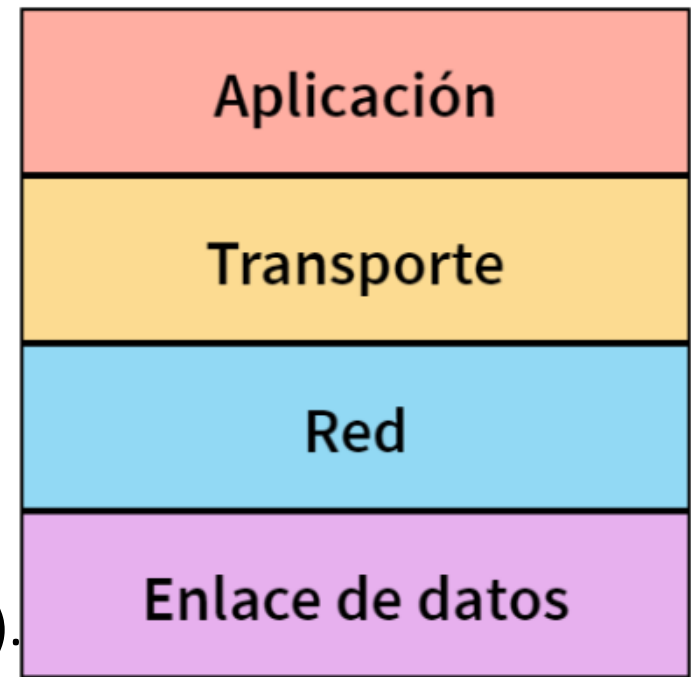
- **El modelo OSI (ISO, 1980)**
 - Modelo de referencia para entender la comunicación en redes.
 - Divide la comunicación en **7 capas**, desde la aplicación hasta el medio físico.
 - Aunque hoy usamos principalmente **TCP/IP**, OSI sigue siendo útil como **guía conceptual**.

LA PILA OSI



EL MODELO TCP/IP

- Modelo de comunicación real de Internet.
- Resume OSI en 4 capas
- Aquí se ubica HTTP en la capa de Aplicación.
- Ejemplos por capa:
 - Aplicación → HTTP (web), correo (SMTP).
 - Transporte → TCP (fiable), UDP (rápido).
 - Red → IP (enrutamiento).
 - Enlace → Ethernet, Wi-Fi.



NIVELES DE ABSTRACCIÓN

EJEMPLO MODELO TCP/IP I

- **Aplicación**

- Envío de solicitud desde el navegador para cargar la página www.ucm.es

- **Transporte**

- La solicitud se divide en paquetes para su transmisión (TCP/UDP)
- Cada paquete tiene una dirección origen y otra destino

- **Red**

- Enrutamiento de los paquetes de datos desde el origen al servidor destino (Protocolo IP)
- Los paquetes se envían a través de routers y switches

- **Enlace de datos**

- Transmisión física de los paquetes de datos a través de medios físicos, cables Ethernet, conexión inalámbrica Wifi.
- Los paquetes se dividen en tramas que contienen información sobre la dirección física de destino

EJEMPLO MODELO TCP/IP II

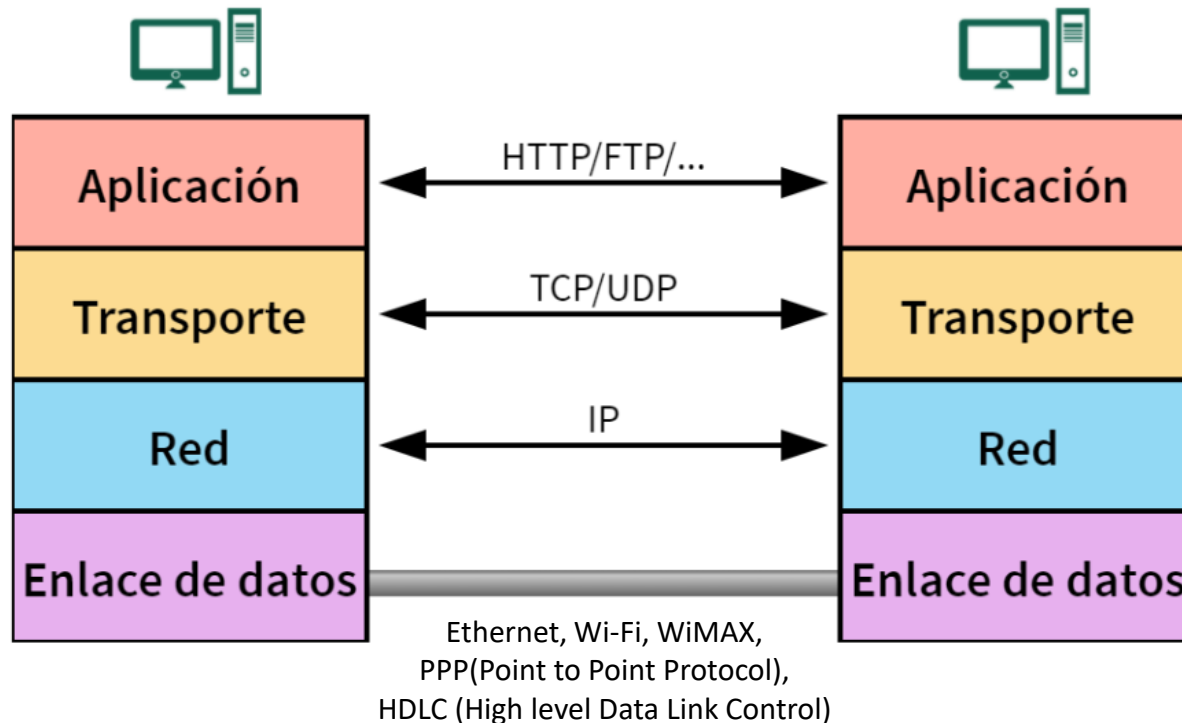
Cuando se solicita una página web en el navegador, este proceso se descompone en cuatro capas del modelo TCP/IP:

1. **Capa de Aplicación:** Tu navegador inicia la solicitud.
2. **Capa de Transporte:** La solicitud se divide en paquetes y se asegura de que lleguen correctamente.
3. **Capa de Red:** Los paquetes se enrutan a través de la red hasta el servidor de destino.
4. **Capa de Enlace de Datos:** Los paquetes se transmiten físicamente a través de la red.

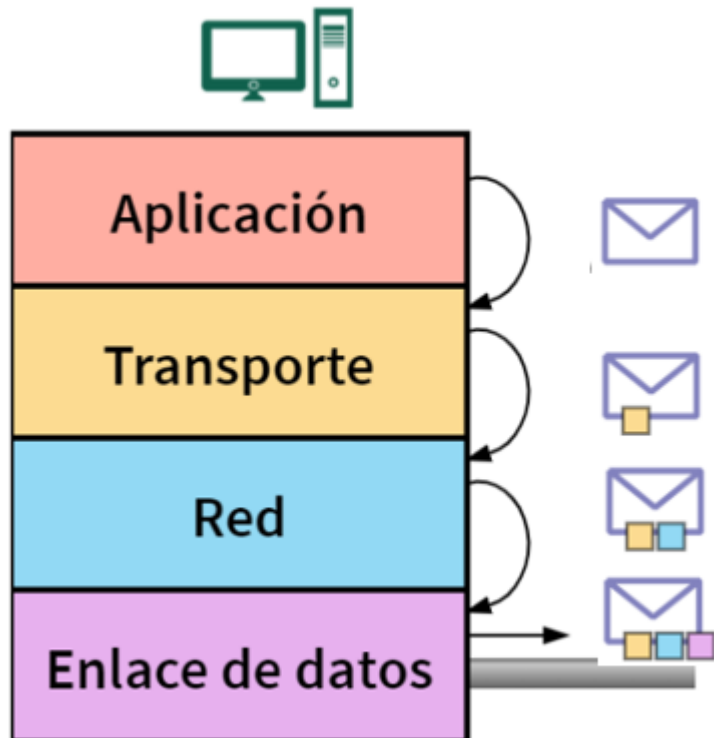
Cada capa desempeña un papel importante en el proceso de comunicación y garantiza que tus datos se transmitan de manera efectiva desde tu computadora al servidor web y viceversa.

EL MODELO TCP/IP

- Cada capa se comunica con su homóloga en otro nodo distinto, utilizando un protocolo como lenguaje



EL MODELO TCP/IP



Sin embargo, la comunicación entre capas homólogas no es directa.

Cada capa solicita a su capa inferior que se envíe el mensaje al destino.

Cada capa extiende el mensaje a enviar con información propia que le permite realizar su cometido.

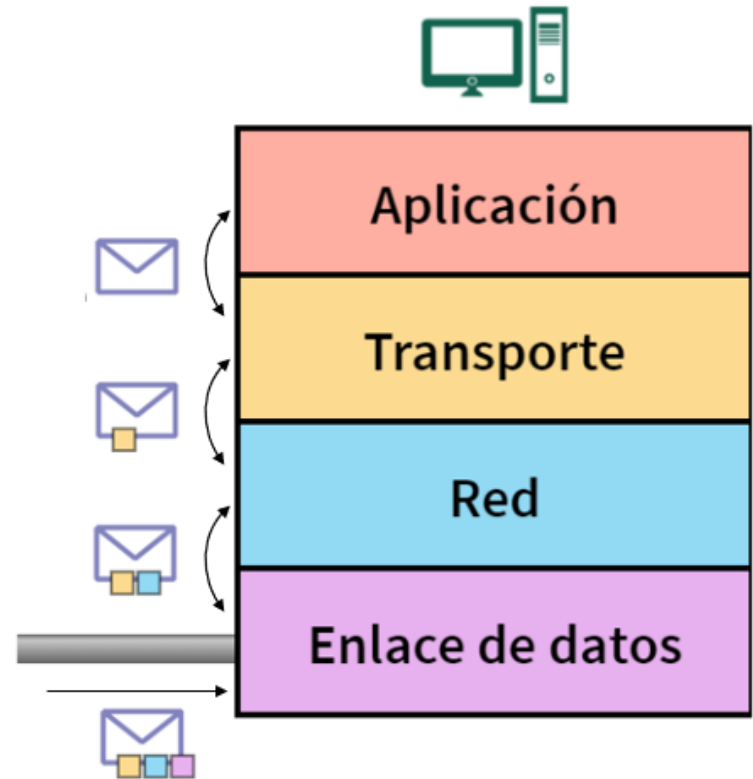
Finalmente, la capa de enlace es la que realiza el envío físico.

EL MODELO TCP/IP

El nodo receptor recibe el mensaje a través de la capa de enlace.

Cada capa interpreta la información adicional introducida por su capa homóloga durante el envío, y acaba remitiendo el mensaje original (sin esta información adicional) a la capa superior.

Finalmente, la capa de aplicación recibe el mensaje tal y como lo envió la capa de aplicación en el origen.



EL MODELO TCP/IP

La capa de enlace de datos proporciona una vía de transmisión de paquetes de datos entre dos nodos enlazados directamente.



EL MODELO TCP/IP

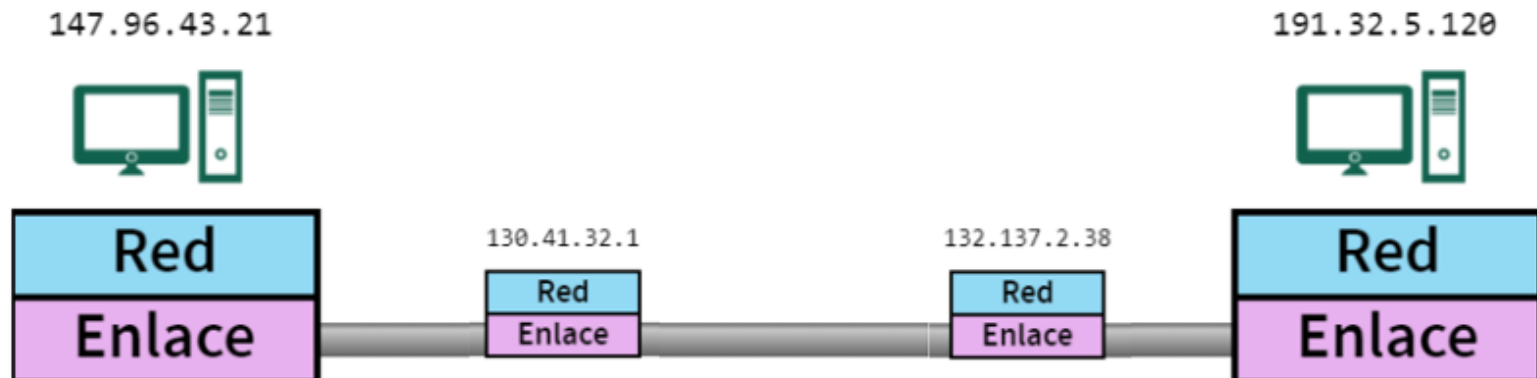
La capa de red gestiona el envío de mensajes a través de múltiples nodos intermedios.

Cada nodo está identificado mediante una
dirección IP.



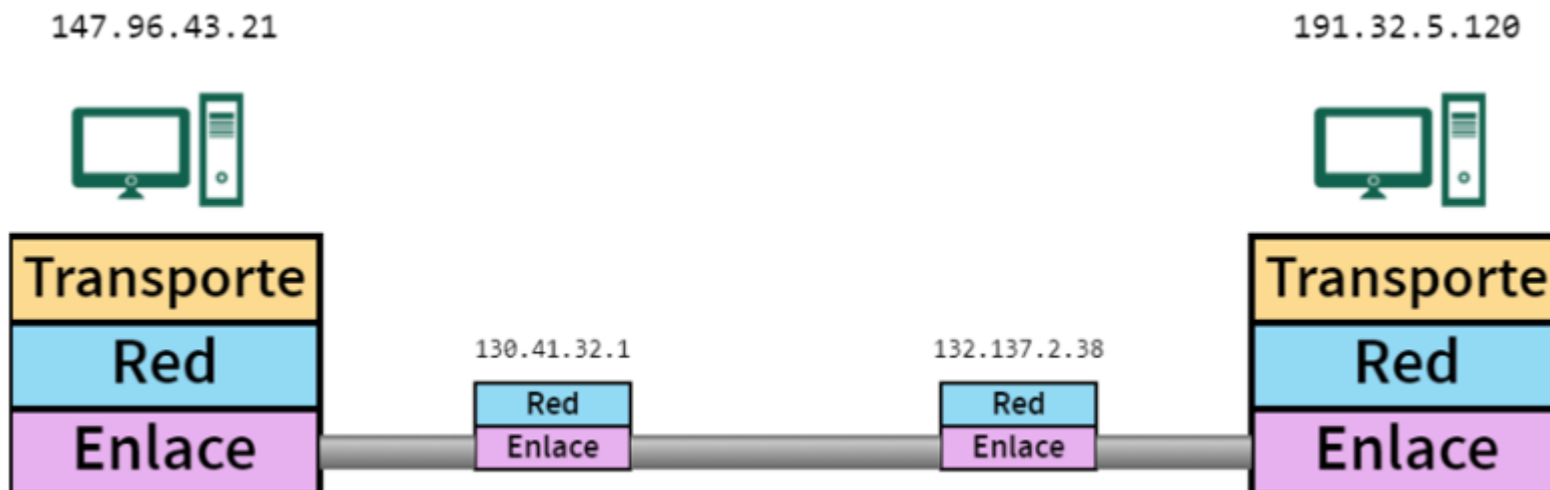
EL MODELO TCP/IP

Las capas de red de los nodos intervinientes enrutan los paquetes desde el origen hasta el destino.



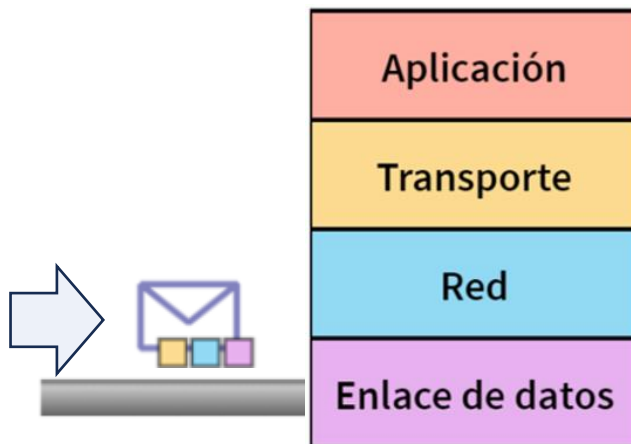
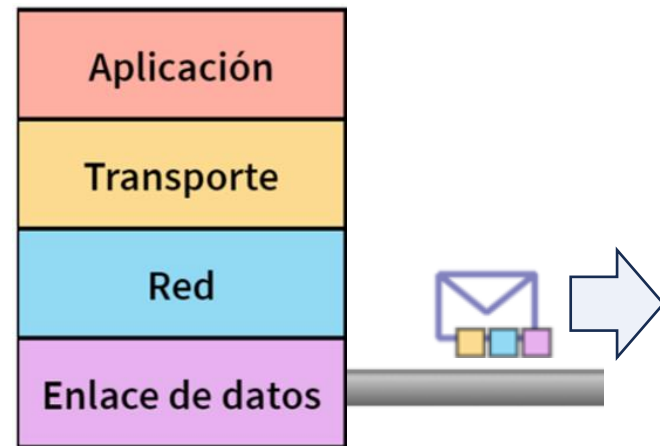
EL MODELO TCP/IP

La capa de transporte se encarga de garantizar la transmisión sin errores desde origen a destino, mediante la segmentación y secuenciación de paquetes de información.



EL MODELO TCP/IP: Ejemplo

- El usuario escribe `www.ucm.es` en el navegador
- Se añade el puerto (TCP 80 o 443)
- Se añade la IP del servidor de `www.ucm.es`.
- se empaqueta en tramas y se envía por la red (WiFi/Ethernet).



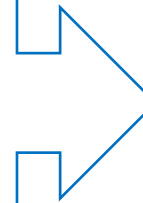
- Entrada de la petición al servidor (Apache, Nginx...) respondiendo con la página solicitada
- Identifica el puerto 80/443 (http-https)
- Lee la dirección IP y confirma que el mensaje es para él.
- El servidor de la UCM recibe la trama en la capa de enlace.

EL MODELO TCP/IP

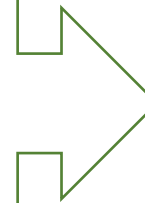
- Proporcionar la transferencia libre de errores de los datos entre emisor-receptor (**Transporte Confiable**)
- Mantener el flujo de la red
- Permite varios canales de comunicación entre nodos
- Cada canal está identificado por un número de puerto.
- Existen puertos TCP y UDP:
 - **TCP (Transmission Control Protocol)**: conexión orientada, bidireccional, fiable (SMTP, POP3, IMAP, FTP, HTTP...). Se establecen tres fases:
 - Establecimiento de conexión
 - Transferencia de datos
 - Liberación
 - **UDP (User Datagram Protocol)**: protocolo mínimo, no conexión, no comprueba duplicado ni extravíos de paquetes, menos confiable (VoIP, Videojuegos en línea, servicios de streaming, IoT...)

EL MODELO TCP/IP

TCP	80	HTTP (Hypertext Transfer Protocol)
	443	HTTPS (HTTP Secure)
	25	SMTP (Simple Mail Transfer Protocol)
	1024::49151	Puertos registrados por IANA
	49152::65535	Puertos dinámicos (asignados por el S.O.)



UDP	53	DNS (Domain Name System)
	67, 68	DHCP (Dynamic Host Configuration Protocol)
	69	TFTP (Trivial File Transfer Protocol)
	123	NDP (Network Time Protocol)
	80, 3000,...	Puertos dinámicos (asignados por S.O.)



CAPA DE
TRANSPORTE

EL MODELO TCP/IP

PUERTOS TCP/UDP MÁS COMUNES

- Puerto 20: **FTP** (transmisión de archivos)
- Puerto 22: **SSH** (transmisión de archivos segura)
- Puerto 25: **SMTP** (envío de correo)
- Puerto 80: **HTTP** (Web)
- Puerto 110: **POP3** (recepción de correo)
- Puerto 143: **IMAP** (recepción de correo)
- Puerto 443: **HTTPS** (Web segura)
- Puerto 995: **POP3S** (recepción de correo segura)
- Puerto 993: **IMAPS** (recepción de correo segura)

Ver: List of TCP and UDP port numbers (Wikipedia)

EJERCICIO: Recorrido de una URL

- Completa qué ocurre en cada capa del modelo TCP/IP cuando se accede a:
 - <https://www.informatica.ucm.es>
 - <https://informatica.ucm.es/ofertas-de-empleo>
 - <https://www.ucm.es/ope/portal-de-empleo-ucm>

Capa de Aplicación

El navegador genera una solicitud _____ a la ruta _____

El protocolo usado es _____

Capa de Transporte

Se abre una conexión _____ con el servidor

Puerto de destino: _____

Capa de Red

El nombre de dominio se resuelve en _____ mediante _____

Los paquetes se enrutan hasta _____

Capa de Enlace

Los datos se transmiten por _____

Se usan direcciones _____ para entregar cada trama

EJERCICIO: Solución

Capa de Aplicación

El navegador genera una solicitud **HTTPS GET** a la ruta **/ofertas-de-empleo**
El protocolo usado es **HTTPS (HTTP sobre TLS)**. **/ope/portal-de-empleo-ucm**

Capa de Transporte

Se abre una conexión **TCP** con el servidor
Puerto de destino: **443**

Capa de Red

El nombre de dominio se resuelve en **una dirección IP** mediante **DNS**
Los paquetes se enrutan hasta **el servidor de la UCM**

Capa de Enlace

Los datos se transmiten por **Ethernet o Wi-Fi**
Se usan direcciones **MAC** para entregar cada trama

CONTENIDO

- EL MODELO TCP/IP
- MODELO CLIENTE/SERVIDOR
- SOCKETS Y PROTOCOLOS
- PROTOCOLO HTTP
- REFERENCIAS



EL MODELO CLIENTE / SERVIDOR

Supongamos que dos nodos quieren conectarse para intercambiar información

¿Cómo y cuándo se establece esta conexión?

En el modelo cliente/servidor cada nodo adquiere un rol:

- **El cliente** que inicia la comunicación enviando información al servidor.
- **El servidor** que permanece conectado a la espera de información que llegue por un determinado puerto. Se dice que el servidor escucha en dicho puerto.

EL MODELO CLIENTE / SERVIDOR

SERVIDOR

- Pasivo: espera a que un cliente envíe una petición
- Responde (sirve) las peticiones de varios clientes.
- A cada tipo de petición se le llama servicio.

CLIENTE

- Activo: toma la iniciativa en la comunicación con el servidor.
- Tiene que conocer la dirección IP del servidor y el puerto al que enviar la información (petición)

EL MODELO CLIENTE / SERVIDOR: Principales características de los sistemas distribuidos

- **Compartición de recursos:**
 - Los servidores gestionan recursos (archivos, servicios, impresoras...).
 - Se encargan de mapear nombres a direcciones y de mantener la consistencia cuando hay accesos concurrentes.
- **Concurrencia:**
 - Varios procesos o usuarios pueden trabajar en paralelo sobre los mismos recursos.
- **Arquitectura abierta:**
 - El sistema permite añadir o ampliar servicios, definiendo hasta dónde puede extenderse.

EL MODELO CLIENTE / SERVIDOR: Principales características de los sistemas distribuidos

- **Escalabilidad:**
 - El sistema se adapta al crecimiento sin necesidad de cambiar aplicaciones ni sistema operativo.
 - Horizontal: añadir o quitar clientes.
 - Vertical: migrar a servidores más potentes o más ligeros.
- **Tolerancia a fallos**
 - Se mantiene el servicio aunque falle un componente, gracias a la redundancia de hardware o software.
- **Transparencia**
 - Para el usuario final, todo el sistema se percibe como un único conjunto integrado, aunque detrás haya muchos equipos.

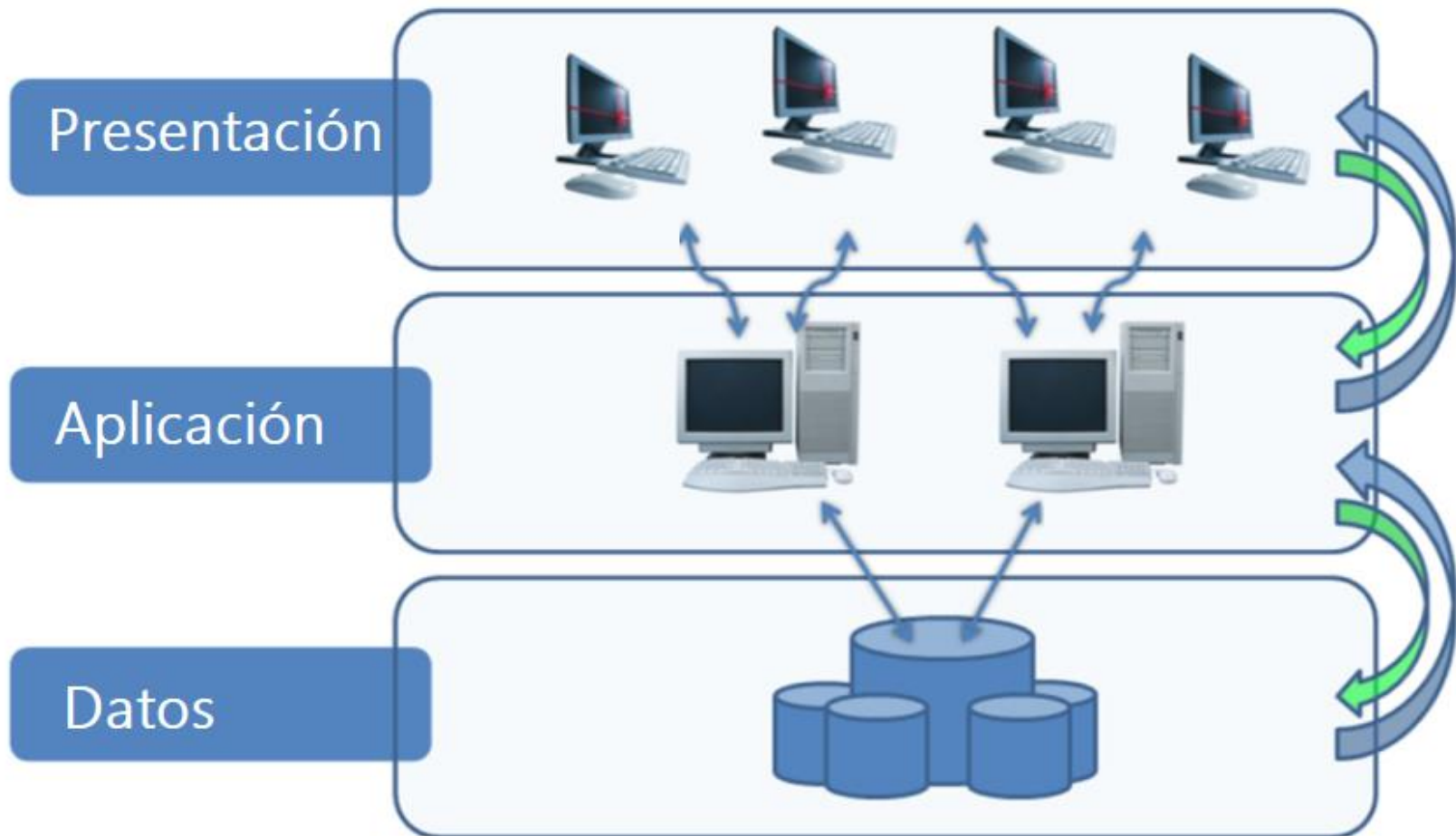
EL MODELO CLIENTE / SERVIDOR:

Clasificación de Aplicaciones

- Aplicaciones de 2 niveles (ejemplo: correo electrónico)
 - Cliente → se encarga de la presentación (interfaz).
 - Servidor → concentra la lógica de negocio y la gestión de datos.
 - ✓ Ventaja: simple.
 - ✗ Inconveniente: difícil de escalar, servidor muy cargado.
- Aplicaciones de 3 niveles (ejemplo: aplicaciones web modernas)
 - Cliente → envía solicitudes y muestra la información.
 - Servidor de Aplicaciones → contiene la lógica de negocio, procesa las solicitudes, pide datos al servidor de BD y envía respuestas.
 - Servidor de Base de Datos → gestiona y almacena todos los datos de la aplicación.
 - ✓ Ventaja: más escalable y flexible.
 - ✗ Inconveniente: mayor complejidad.

MODELO CLIENTE / SERVIDOR

Arquitectura de 3 capas



CONTENIDO

- EL MODELO TCP/IP
- MODELO CLIENTE/SERVIDOR
- SOCKETS Y PROTOCOLOS
- PROTOCOLO HTTP
- REFERENCIAS



SOCKETS Y PROTOCOLOS: Definición

Supongamos que un servidor está escuchando en un determinado puerto y que un cliente se conecta al mismo:

[Cliente (Navegador)]		[Servidor Web]
IP: 192.168.1.35:52341	----->	IP: 150.244.100.20:80
Puerto aleatorio		Puerto 80 (HTTP)

Esto crea un canal de comunicación bidireccional entre el cliente y el servidor.

A cada uno de los extremos de este canal de comunicación se le llama *socket*

Un socket es el mecanismo que nos da el SO para acceder a la red

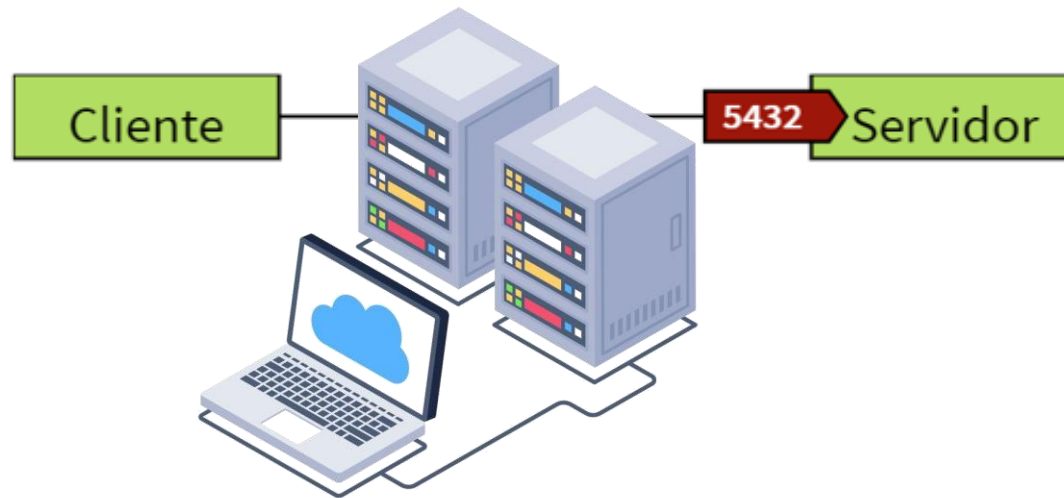
SOCKETS Y PROTOCOLOS: Flujos

Cada socket tiene asociados dos flujos:

- **Flujo de entrada** (*input stream*)
La información que llegue desde el otro extremo se lee a través de este flujo.
- **Flujo de salida** (*output stream*)
Todo lo que se escriba en este flujo es enviado al otro extremo.

Cuando en el navegador escribes en un formulario, los datos van por el flujo de salida; la respuesta del servidor va por el flujo de entrada.

SOCKETS Y PROTOCOLOS: Tipos



SOCKET DE SERVIDOR

- Recibe las peticiones a través de su flujo de entrada.
- Envía las respuestas a través del flujo de salida.

SOCKET DE CLIENTE

- Envía las peticiones a través del flujo de salida.
- Recibe las respuestas a través del flujo de entrada.

SOCKET Y PROTOCOLOS: APLICACIONES WEB



Las aplicaciones web también utilizan el modelo cliente-servidor:

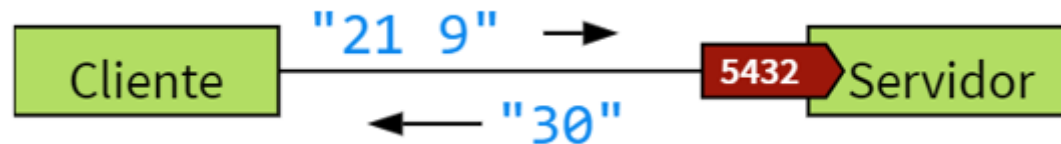
- **Cliente:** Navegador web.
Firefox, Chrome, Opera, IE
- **Servidor:** Servidor web
Apache HTTP Server, Tomcat, NginX, etc...
- **Protocolo:** HTTP / HTTPS



SOCKETS Y PROTOCOLOS: Ejemplo 1

EJEMPLO: SERVIDOR CALCULADORA

Implementemos en Java un sencillo servidor que realiza la suma de dos números.



El cliente envía al servidor una cadena de texto con dos números enteros.

El servidor responde con la suma de dichos números.

EJEMPLO 1: Servidor

IMPLEMENTACIÓN EN EL SERVIDOR

```
import java.net.ServerSocket;
import java.net.Socket;

public class ArithServer {
    // Número de puerto
    private static final int PORT_NUMBER = 5432;

    public static void main(String[] args) {
        try {
            ServerSocket server = new ServerSocket(PORT_NUMBER);
            Socket s = server.accept();
            serverPetition(s); // Implementado más adelante.
        } catch (IOException ex) {
            ...
        }
    }

    private static void servePetition(Socket s) { ... }
}
```

EJEMPLO 1: Servidor

```
ServerSocket server = new ServerSocket(PORT_NUMBER);
```

Crea un objeto **ServerSocket**, que permite recibir conexiones a través del puerto dado.

```
Socket s = server.accept();
```

Queda a la espera de que se conecte un cliente.

El método **accept()** **detiene** la ejecución del programa hasta que reciba la conexión de un cliente.

El objeto de la clase Socket es el que permite acceder a los flujos de entrada y salida del socket.

```
public InputStream getInputStream();  
public OutputStream getOutputStream();
```

EJEMPLO 1: Cliente

IMPLEMENTACIÓN EN EL CLIENTE

```
public static void main(String[] args) {  
    try {  
        Socket s = new Socket("localhost", PORT_NUMBER);  
        // Enviar información al servidor y recibir respuesta  
        int res = add(s, 21, 45);  
        System.out.println("21 + 45 = " + res);  
    } catch (IOException ex) {  
        System.out.println("Error de conexión: " + ex.getMessage());  
    }  
}
```

La conexión a un servidor se realiza creando directamente una instancia de la clase **Socket**, indicando el servidor al que conectarse (**localhost** = la propia máquina) y el puerto.

EJEMPLO 1: Operaciones

La función **add()** es la que envía la petición al servidor:

```
private static int add (Socket s, int n1, int n2) {
    try (PrintWriter out =
        new PrintWriter(new OutputStreamWriter(s.getOutputStream()));
        Scanner sc = new Scanner(s.getInputStream())) {
        // Enviar sumandos al servidor
        out.print(n1);
        out.print(" ");
        out.print(n2);
        out.flush();
        //Recibir el resultado
        return sc.nextInt();
    } catch (IOException ex) {
        System.err.println("Error de E/S: " + e.getMessage());
    }
}
```

EJEMPLO 1

Para ejecutar este ejemplo:

- Ejecutamos el método **main()** del servidor.
- Ejecutamos el método **main()** del cliente, obteniendo como salida:

```
21 + 45 = 66
```

- Tras esto, tanto el cliente como el servidor **finalizan**.

EJEMPLO 1: Bucle

Si queremos que el servidor siga atendiendo peticiones de otros clientes, podemos hacerlo mediante un bucle que se repita indefinidamente:

```
ServerSocket server = new ServerSocket(PORT_NUMBER);  
    while (true) {  
        Socket s = server.accept();  
        serverPetition(s);  
    }
```

EJEMPLO 1: Operaciones disponibles

EXTENSIÓN DEL SERVIDOR

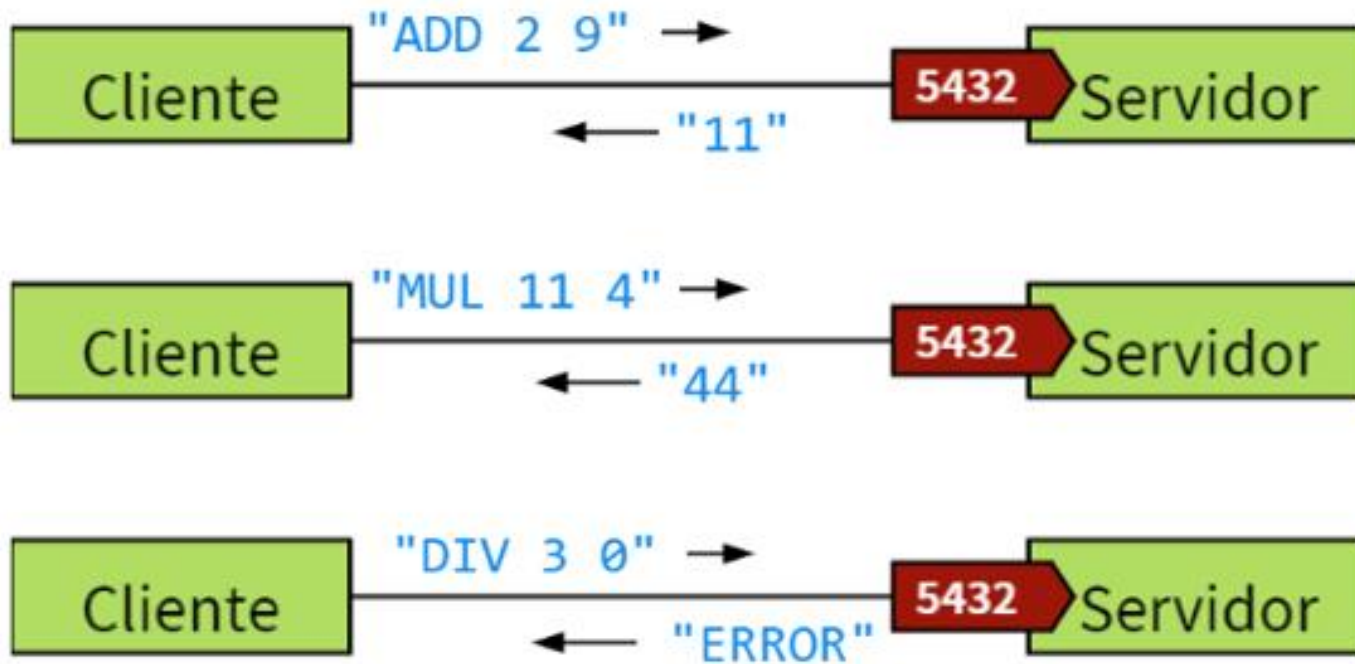
Modificamos el servidor para que pueda realizar otro tipo de operaciones aritméticas.

El servidor recibirá mensajes de la forma:

- ADD n1 n2
- SUB n1 n2
- MUL n1 n2
- DIV n1 n2
- SQRT n1

EJEMPLO 1: Operaciones disponibles

EJEMPLOS



EJEMPLO 1: Servidor

CÓDIGO DEL SERVIDOR

```
private static void serverPetition(Socket socket) {
    try (Scanner sc = new Scanner(socket.getInputStream());
        PrintWriter out = new PrintWriter(
            new OutputStreamWriter(socket.getOutputStream())
        )) {
        String op = sc.next();
        switch (op) {
            case "ADD" :    serveAddition(sc, out); break;
            case "SUB" :    serveSubstraction(sc, out); break;
            case "MUL" :    serveMultiplication(sc, out); break;
            case "DIV" :    serveDivision(sc, out); break;
            case "SQRT" :   serveSquareRoot(sc, out); break;
            default :       out.println("ERROR");
        }
    } catch (IOException e) {
        System.err.println("Error al obtener el flujo de entrada: " + e.getMessage());
    }
}
```

EJEMPLO 1: Nueva operación en Servidor

EJEMPLO: DIVISIÓN

```
private static void serveDivision(Scanner sc, PrintWriter out) {  
    int n1 = sc.nextInt();  
    int n2 = sc.nextInt();  
    if (n2 != 0) {  
        System.out.println("Dividiendo " + n1 + " entre " + n2);  
        out.println((double)n1 / n2);  
    } else  
        out.println("ERROR");  
}
```

EJEMPLO 1: Cliente

PETICIÓN DEL CLIENTE

```
public static void main(String... args) throws IOException {  
    Socket s = new Socket("localhost", 5432);  
    BufferedWriter bw = new BufferedWriter(  
        new OutputStreamWriter(s.getOutputStream()));  
    bw.write("MUL 4 12\n");  
    bw.flush();  
    BufferedReader br = new BufferedReader(  
        new InputStreamReader(s.getInputStream()));  
    System.out.println(br.readLine());  
    bw.close();  
    br.close();  
}
```

Resultado:

EJEMPLO 1: Cliente

```
import java.io.*;
import java.net.*;

public class ClienteMultiplicacion {
    public static void main(String[] args) {
        try {
            // Establece la conexión con el servidor en localhost y el puerto 5432.
            Socket socket = new Socket("localhost", 5432);
            // Establece flujos de entrada y salida para la comunicación con el servidor.
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            BufferedReader userInput = new BufferedReader(new InputStreamReader(System.in));
            // Lee dos números desde el usuario.
            System.out.print("Ingrese el primer número: ");
            double numero1 = Double.parseDouble(userInput.readLine());
            System.out.print("Ingrese el segundo número: ");
            double numero2 = Double.parseDouble(userInput.readLine());
            // Envía los números al servidor.
            out.println(numero1);
            out.println(numero2);
            // Lee la respuesta del servidor (resultado de la multiplicación).
            String resultado = in.readLine();
            System.out.println("Resultado de la multiplicación: " + resultado);
            // Cierra los flujos y el socket.
            out.close();
            in.close();
            userInput.close();
            socket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

SOCKETS Y PROTOCOLOS

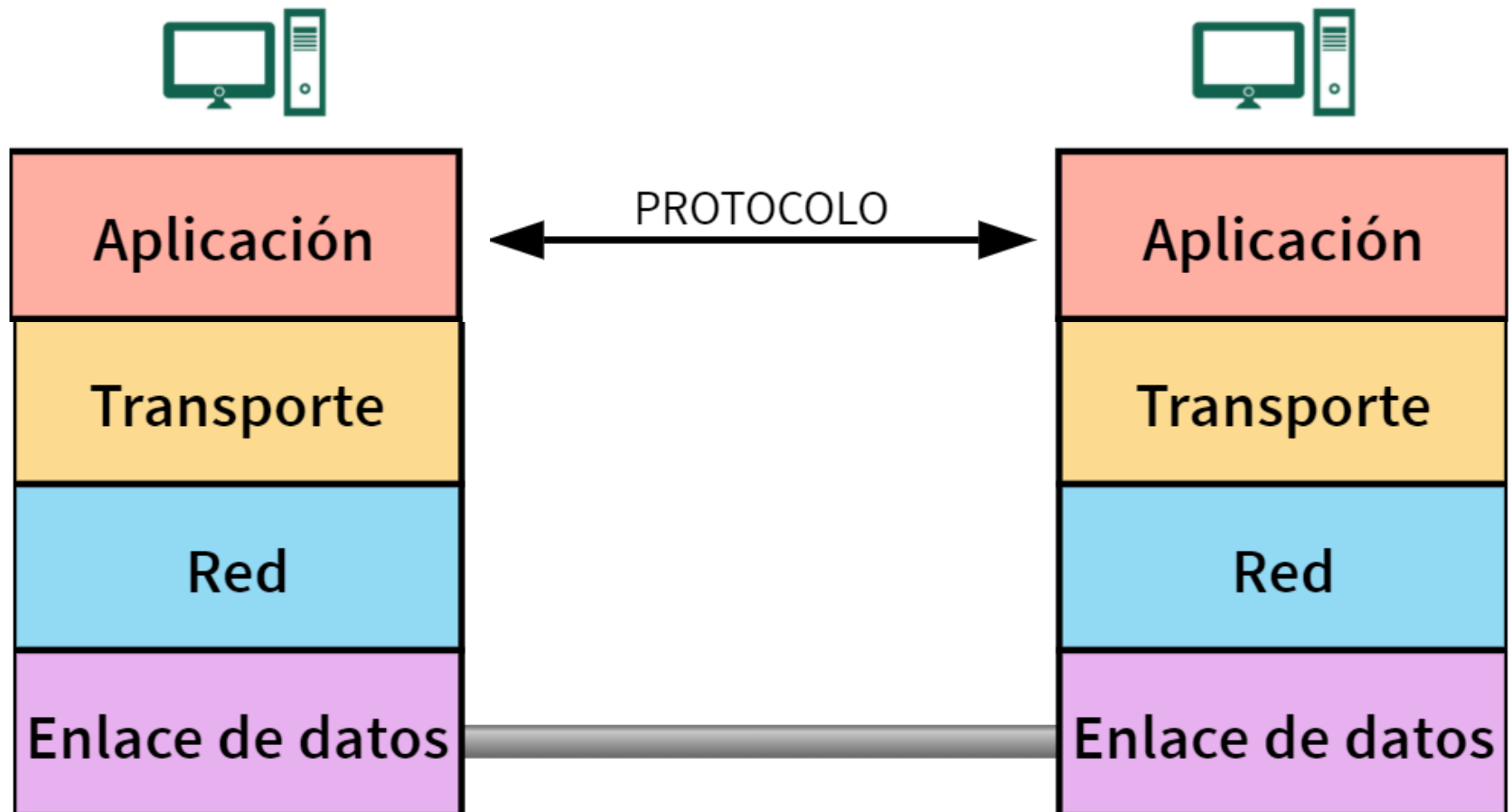
RESUMEN: COMUNICACIÓN CON EL SERVIDOR

Petición	Respuesta
ADD $n1$ $n2$	$n1 + n2$
SUB $n1$ $n2$	$n1 - n2$
MUL $n1$ $n2$	$n1 * n2$
DIV $n1$ $n2$	$n1 / n2$ (si $n1 \neq 0$) ERROR (e.o.c.)
SQRT n	$\sqrt{n}$ (si $n \geq 0$) ERROR (e.o.c.)

Esta tabla describe un **PROTOCOLO**

SOCKETS Y PROTOCOLOS

Hemos implementado *la capa de aplicación* de la calculadora



EXPERIMENTO: SOCKET & PROTOCOLO

Conectamos con el servidor de la Fdl por protocolo HTTP 1.0 y puerto 80

```
import java.io.*;
import java.net.*;

public class informaticaUCM {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("informatica.ucm.es", 80);
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(s.getOutputStream()));
            // Petición HTTP 1.1
            pw.println("GET / HTTP/1.1");
            pw.println("Host: informatica.ucm.es");
            pw.println("Connection: close"); // Indica que se cerrará la conexión después de recibir la respuesta
            pw.println();
            pw.flush(); // Importante: asegura que los datos se envíen al servidor
            // Respuesta
            BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
            String line;
            while ((line = br.readLine()) != null)
                System.out.println(line);
            // Cierra los flujos y el socket
            pw.close();
            br.close();
            s.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

EXPERIMENTO: SOCKET & PROTOCOLO

Respuesta esperada

```
HTTP/1.1 200 OK
Date: Wed, 10 Aug 2016 16:40:14 GMT
Server: Apache
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0,
               pre-check=0
Pragma: no-cache
Content-Type: text/html; charset=UTF-8
Set-Cookie: ucmweb=7rbkr7h03pld7rkugaod7u28a6; path=/;
            domain=ucm.es
Connection: close

<!DOCTYPE html>
<html lang="es">
<head>
    <title>Facultad de Informática. Universidad Complutense de
    ...
```


EXPERIMENTO: SOCKET & PROTOCOLO

Respuesta obtenida

```
HTTP/1.0 302 Moved Temporarily  
Location: https://informatica.ucm.es/  
Server: BigIP  
Connection: close  
Content-Length: 0
```

EXPERIMENTO: SOCKET & PROTOCOLO

Conectamos con el servidor de la Fdl en el puerto 443 (SSL)

```
import java.io.*;
import java.net.*;
import javax.net.SocketFactory;
import javax.net.ssl.SSLSocket;
import javax.net.ssl.SSLSocketFactory;
public class informaticaUCM {
    public static void main(String[] args) {
        try {
            // Crear un socket SSL para HTTPS en el puerto 443
            SocketFactory factory = SSLSocketFactory.getDefault();
            SSLSocket s = (SSLSocket) factory.createSocket("informatica.ucm.es", 443);
            // Establecer flujos de entrada y salida para la comunicación con el servidor
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(s.getOutputStream()));
            // Petición HTTP 1.1
            pw.println("GET / HTTP/1.1");
            pw.println("Host: informatica.ucm.es");
            pw.println("Connection: close"); // Indica que se cerrará la conexión después de recibir la respuesta
            pw.flush(); // Asegura que los datos se envíen al servidor
            // Respuesta
            BufferedReader br = new BufferedReader(new InputStreamReader(s.getInputStream()));
            String line;
            while ((line = br.readLine()) != null)
                System.out.println(line);
            // Cierra los flujos y el socket
            pw.close(); br.close(); s.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

EXPERIMENTO: SOCKET & PROTOCOLO

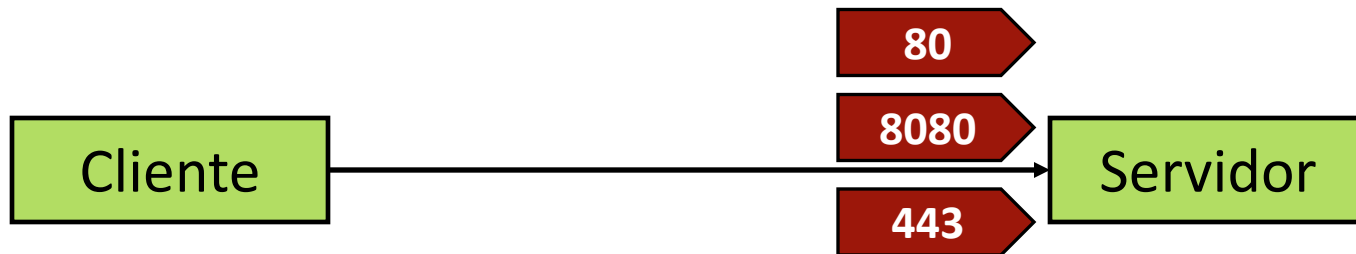
Respuesta obtenida:

```
HTTP/1.1 200 OK
Date: Sat, 04 Sep 2021 09:05:14 GMT
Server: Apache
Set-Cookie: ucmweb=2gljvc5tdui9sk3jef5oq5f8c2vtg625sps63ms7qa2qpuipi141; path=/;
domain=.ucm.es; secure; HttpOnly
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Connection: close
Content-Type: text/html; charset=UTF-8
Strict-Transport-Security: max-age=63072000
Vary: Accept-Encoding
```

```
2000
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
<title>Facultad de Informática</title>...
```

SOCKETS Y PROTOCOLOS

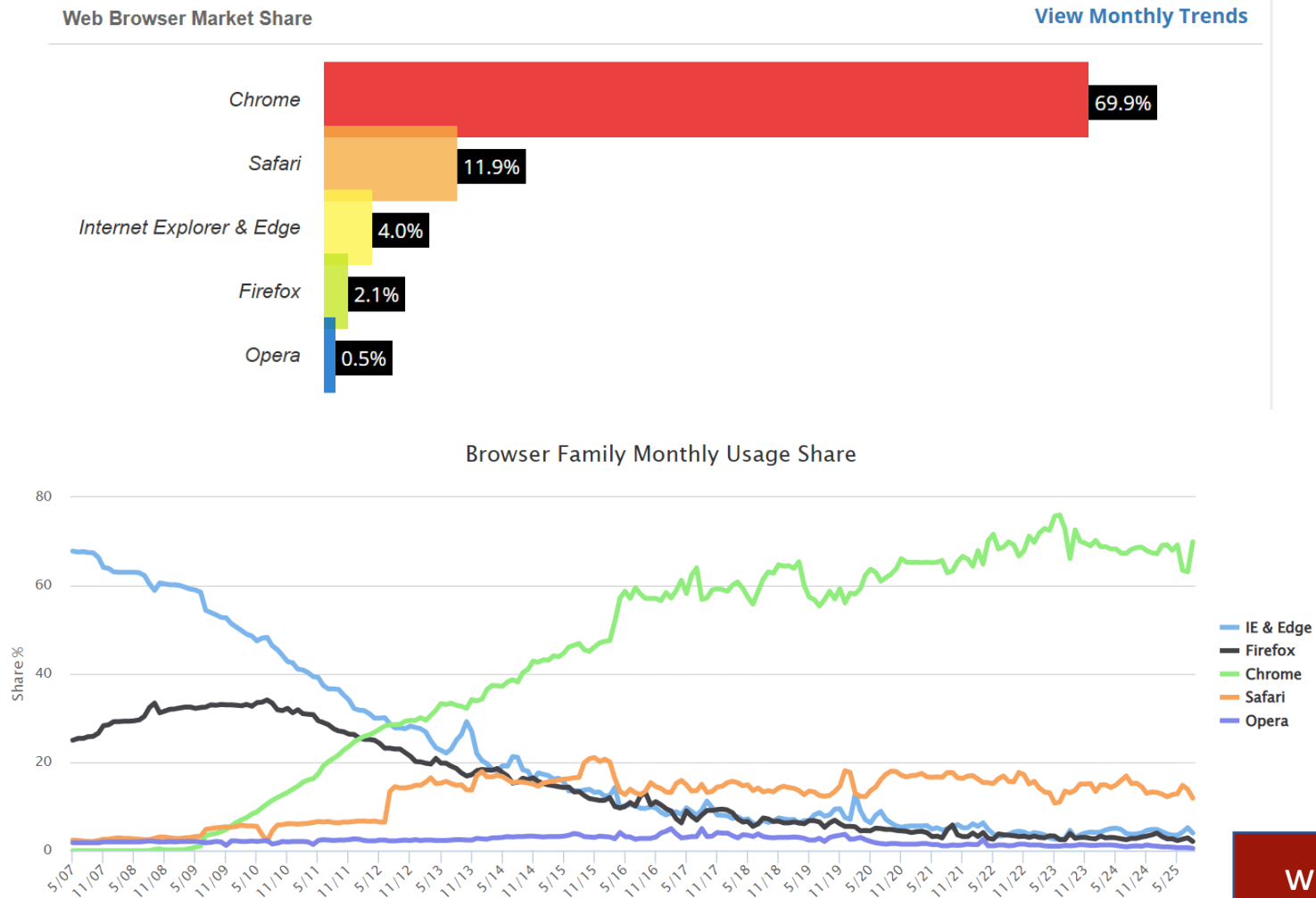
- Las aplicaciones Web también utilizan el modelo cliente-servidor:



- Cliente: Navegador web
Firefox, Chrome, Opera, Explorer, etc
- Servidor:
Apache HTTP server, Tomcat, Microsoft IIS, etc.
- Protocolo: HTTP (HTTPS)

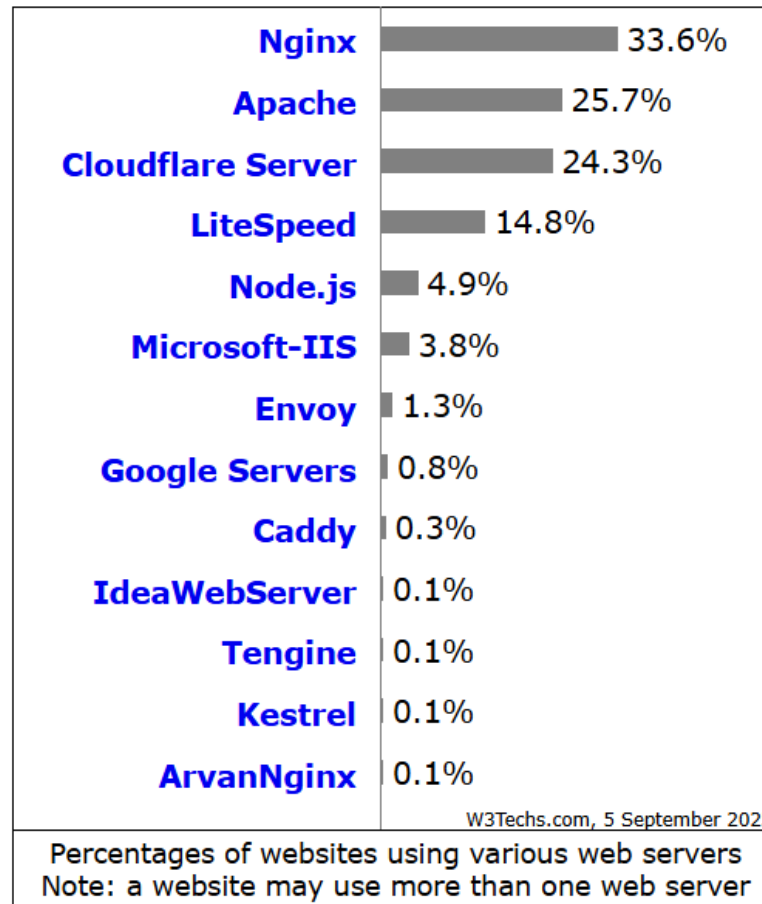
SOCKETS Y PROTOCOLOS

ESTADÍSTICAS NAVEGADORES WEB



SOCKETS Y PROTOCOLOS

ESTADÍSTICAS SERVIDORES WEB



SOCKETS Y PROTOCOLOS: Seguridad

- **SSL (Secure Sockets Layer)**
 - Mantiene segura una conexión a Internet
 - Proteger cualquier información enviada a otro sistema
- **TLS (Transport Layer Security)**
 - Versión actualizada de SSL (ECC, RSA, DSA)
- **HTTPS (Hyper Text Transfer Protocol Secure)**
 - Aparece en la URL del navegador
 - Indica que el sitio web está protegido por un certificado SSL



CONTENIDO

- EL MODELO TCP/IP
- MODELO CLIENTE/SERVIDOR
- SOCKETS Y PROTOCOLOS
- PROTOCOLO HTTP
- REFERENCIAS

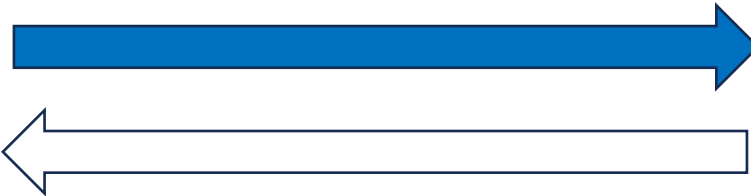


PROTOCOLO HTTP: como hablan navegadores y servidores

C
L
I
E
N
T
E
S



GET /index.html HTTP/1.1



HTTP/1.1 200 OK



S
E
R
V
I
D
O
R
E
S

```
GET /register.html HTTP/1.1
```

```
Host: www.foo.com
```

```
Accept: text/html
```

```
Accept-Language: en-US, es-ES
```

```
Connection: keep-alive
```

```
If-Modified-Since: Fri, 26 Feb 2016 03:09:05 GMT
```

(línea en blanco)

```
name=Francisco%20Sanchez&age=12
```

```
...
```

Línea de petición

Cabecera

Cuerpo de la petición (opcional)

PROTOCOLO HTTP

LÍNEA DE PETICIÓN

Tiene el siguiente formato:

```
[método] [recurso] [versión_HTTP]
```

- **[método]**: Acción a realizar (GET, POST, HEAD, PUT, ...)
- **[recurso]**: Nombre del recurso (URI) sobre el que se accederá.
- **[versión_HTTP]**: HTTP/1.0 o HTTP/1.1.

Ejemplo:

```
GET /index.html HTTP/1.1
```

PROTOCOLO HTTP

ALGUNOS MÉTODOS DISPONIBLES

- **GET**: Obtener un recurso del servidor.
- **HEAD**: Igual que GET, pero el servidor no devolverá el recurso; sólo las cabeceras de la respuesta.
- **POST**: Enviar información al servidor.
- **PUT**: Guardar información en el servidor.
- **DELETE**: Eliminar información del servidor.
- **OPTIONS**: Obtener la lista de métodos soportados por el servidor.

PROTOCOLO HTTP

RECURSOS HTTP

```
[método] [recurso] [versión_HTTP]
```

El nombre de recurso tiene la apariencia de un nombre de fichero, pero **no tiene por qué corresponder con un fichero físico almacenado en el servidor**

Un servidor web puede hacer corresponder las URIs con llamadas a programas (por ejemplo, servlets), accesos a recursos físicos almacenados con un nombre distinto, etc.

PROTOCOLO HTTP

CABECERAS EN UNA PETICIÓN

Especifican información adicional en una petición

```
Host: www.foo.com  
Accept: text/html  
Accept-Language: en-US, es-ES  
Connection: keep-alive  
If-Modified-Since: Fri, 26 Feb 2016 03:09:05 GMT
```

PROTOCOLO HTTP

ALGUNOS EJEMPLOS DE CABECERAS

- **Host:** `www.ucm.es`
Obligatorio en HTTP/1.1. Indica el servidor virtual al que acceder.
- **Accept:** `text/html, image/gif, */*`
Tipos de contenido (MIME types) que se espera recibir ([Lista de tipos MIME](#))
- **Accept-Language:** `en-US`
Idioma(s) en los que se espera recibir el recurso ([Idiomas y Regiones](#))
- **Accept-Charset:** `UTF-8`
Accept-Encoding: `x-gzip`
Codificación de caracteres.

PROTOCOLO HTTP

ALGUNOS EJEMPLOS DE CABECERAS

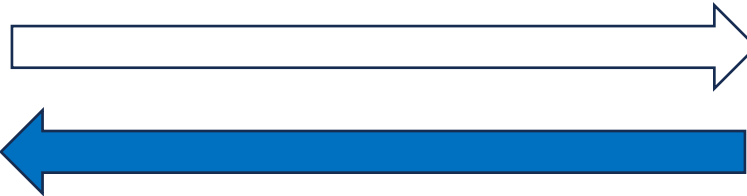
- **User-Agent: Mozilla/5.0 ...**
Navegador o cliente que envía la petición.
- **Content-Type: text/html**
En peticiones POST, indica el tipo de contenido que se adjunta en el cuerpo de la petición.
- **Authorization: Basic bWFudWVsOm1hbnVlbA==**
Nombre de usuario y contraseña (para acceso a recursos protegidos por contraseña).
- **If-Modified-Since: Fri, 26 Feb 2016 03:09:05 GMT**
Indica al servidor que sólo devuelva el recurso si ha sido modificado después de la fecha dada.

PROTOCOLO HTTP: como hablan navegadores y servidores

C
L
I
E
N
T
E
S



GET /index.html HTTP/1.1



HTTP/1.1 200 OK



S
E
R
V
I
D
O
R
E
S

HTTP/1.1 200 OK

Date: Mon, 15 Aug 2016 14:05:30 GMT

Server: Apache

Set-Cookie: ucmweb=a8bcbkqfml72df71mee86drgj7; domain=ucm.es

Expires: Thu, 19 Nov 1981 08:52:00 GMT

Content-Type: text/html; charset=utf-8

(línea en blanco)

<!DOCTYPE html>

<html lang="es">

<head>

<title>UCM-Universidad Complutense de Madrid</title>

...

Línea de estado

Cabecera de la respuesta

Cuerpo de la respuesta

PROTOCOLO HTTP

LÍNEA DE ESTADO

```
HTTP/1.1 200 OK
```

Indica la versión del protocolo HTTP utilizada, un código de estado y un texto con la descripción dicho código.

PROTOCOLO HTTP

CÓDIGOS DE ESTADO

- **200 OK**: Petición recibida y servida con éxito.
- **3xx**: Redirección: el recurso ha cambiado de dirección.
 - **301 Move Permanently**
 - **302 Move Temporarily**
 - **304 Not modified**

En los códigos 301 y 302 se indica en la cabecera de la respuesta la nueva dirección del recurso. El código 304 se devuelve en el caso en el que el cliente haya incluido **If-Modified-Since** y el recurso no haya cambiado desde la fecha indicada.

PROTOCOLO HTTP

CÓDIGOS DE ESTADO

- **4xx**: Errores atribuibles al cliente.
 - **400 Bad Request**: Petición incorrecta.
 - **401 Authentication Required**: Se requiere identificación (nombre y contraseña).
 - **403 Forbidden**: Identificación incorrecta.
 - **404 Not Found**: Recurso no encontrado en el servidor.
 - **405 Method not Allowed**: Acción (GET, POST, etc.) no permitida.
- **5xx**: Errores atribuibles al servidor.
 - **500 Internal Server Error**
 - **503 Service Unavailable**

PROTOCOLO HTTP

CUERPO DE LA RESPUESTA

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>UCM-Universidad Complutense de Madrid</title>
...
```

Tal y como indica la cabecera **Content-Type: text/html**, el cuerpo de la respuesta contiene un texto en formato HTML que define la estructura de la página web recuperada.

PROTOCOLO HTTP

El código HTML puede hacer referencia a otros recursos:

```
...  
<link type="text/css" media="screen" rel="stylesheet"  
href="/themes/ucm3/css/portada.css?a=1" />  
...  
  
...
```

Cada uno de estos recursos deberá ser obtenido por el navegador Web en sucesivas peticiones HTTP al servidor.

PROTOCOLO HTTP

VISUALIZAR CÓDIGO HTML EN FIREFOX Y CHROME

- Botón derecho → *Ver código fuente*, o bien,
- Ctrl+U



PROTOCOLO HTTP: Código fuente

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <title>Universidad Complutense de Madrid</title>
8
9   <meta name="description" content="La mayor universidad presencial de España, con la mejor oferta educativa. En Madrid, rodeada de zonas verdes y con una int<
10  <meta name="keywords" content="Universidad Complutense de Madrid, Complutense, UCM, Educación, Innovación, Universidad de excelencia, Formación, Grado, Máster<
11  <link rel="apple-touch-icon" sizes = "192x192" href="/themes/ucm24/media/img/favicon-192.png">
12  <link rel="shortcut icon" href="/themes/ucm24/media/img/logoucm.ico">
13  <link rel="icon" href="/themes/ucm24/media/img/logoucm.ico">
14  <link type="text/css" media="screen" rel="stylesheet" href="/themes/ucm24/css/bootstrap.css">
15  <link type="text/css" media="all" rel="stylesheet" href="/themes/ucm24/css/font-awesome.min.css">
16  <link type="text/css" media="screen" rel="stylesheet" href="/themes/ucm24/css/ucm-theme.css?ft=20250905132549">
17  <link type="text/css" media="screen" rel="stylesheet" href="/themes/ucm24/css/ucm.css?ft=20250905132549">
18  <link type="text/css" media="screen" rel="stylesheet" href="/themes/ucm24/css/ucm-wg.css?ft=20250905132549">
19  <link type="text/css" media="print" rel="stylesheet" href="/themes/ucm24/css/print.css?ft=20250905132549">
20  <link type="text/css" media="screen" rel="stylesheet" href="/themes/ucm24/css/ucm_home.css?ft=20250905132549">
21 <link type="text/css" media="all" rel="stylesheet" href="/themes/ucm24/css/menu_3.css?ft=20250905132548">
22 </head>
23 <body>
24   <header>
25     <div class="container-fluid" id="cabecera">
26       <div id="logo">
27         <a href="https://www.ucm.es/">
28           
29         </a>
30       </div>
31       <div id="titulo_website">
32         <h1 class="web_title_ucm"><a href="https://www.ucm.es/" title="Portada - Universidad Complutense de Madrid">Universidad Complutense de Madrid</a>
33       <div id="cab-zona-1">
34         <div id="cab-sub-2">
35           <button type="button" class="navbar-toggle" data-toggle="collapse" data-target=".navbar-sup-collapse" id="collapse-personal">
36             <span class="sr-only">Desplegar navegación</span>
37             <span class="fa fa-user fa-2x"></span>
```

PROTOCOLO HTTP

OBSERVAR CABECERAS HTTP EN CHROME

Menú *Más Herramientas* → *Herramientas para desarrolladores* (Ctrl+Mayus+I)



<https://youtu.be/B4vQHymBP1A>

CONTENIDO

- EL MODELO TCP/IP
- MODELO CLIENTE/SERVIDOR
- SOCKETS Y PROTOCOLOS
- PROTOCOLO HTTP
- REFERENCIAS



REFERENCIAS

- Apuntes de otras asignaturas sobre protocolo HTTP
- Wikipedia – Internet Protocol Suite
 - https://en.Wikipedia.org/wiki/Internet_protocol_suite
- *In Introduction to HTTP Basics*
 - https://www3.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html